Database Systems

First Semester 2026-27

A Project Report on

Design and Implementation of a Database-Backed Music Streaming Platform

Mid-Semester Submission

Database Design and Application Design

Submitted by

Ritwik Sethi	2023B1A71385H
Arihant Rakhecha	2023B1A70998H
Shreyas Ghosh	2024A1PS0826H
Shreya Agrawal	2023B5A30794H
Piyush Gaikwad	2024A1PS0859H

Birla Institute of Technology and Science, Pilani

Hyderabad Campus

October 2026

PART A: DATABASE DESIGN DOCUMENT

1 Requirements Analysis

1.1 Problem and users

We are building the database behind a music streaming platform. It keeps track of a music catalogue (artists, albums, songs and who performed on each song) and of what users do with it: their playlists, liked songs, listening history and subscription. Since almost every piece of data points at another, we want the database itself to guarantee consistency. The same data also has to answer analytical questions, such as which songs are trending in a country.

The system has three kinds of users. A *listener* searches the catalogue, plays and likes songs, builds playlists, manages a subscription and looks at charts and a personal "Wrapped" summary. An *artist* is a user account tied to one artist profile; through it the artist releases albums and sees statistics for their own songs. An *admin* maintains users and the catalogue and views platform-wide reports. Actual audio streaming, real payment processing and recommendations are outside the scope of this project.

1.2 Assumptions

1. Every album is released by exactly one artist. Other artists who appear on a track are credited per song, through *Performs*.
2. A single is treated as an album whose type is `'single'`, so a song's release date is simply the release date of its album.
3. A user has at most one subscription record. A user without one is on the free tier.
4. Each artist has one genre. An artist profile is managed by at most one user account, and a user manages at most one profile.
5. One listening event is identified by who played it, which song it was and when it was played.
6. Users must be at least 13 years old, and free users can own at most five playlists.

1.3 Business rules

ID	Rule	How it is enforced
BR1	No duplicate likes, and no song twice in a playlist	Composite primary key
BR2	Every album has an artist, every song an album, every playlist an owner	NOT NULL foreign key
BR3	Every song has at least one `'Lead'` performer	`release_album()` transaction
BR4	No listen before the album's release or in the future	Trigger T1
BR5	No song added to a playlist before the playlist existed	Trigger T2
BR6	A song's play count equals its number of listens	Trigger T3
BR7	Users are at least 13; subscriptions cannot start in the future	Trigger T4
BR8	Without an active paid subscription, at most five playlists	Trigger T5
BR9	Account type is `'artist'` exactly when an artist profile is linked	CHECK
BR10	Valid date ranges, status values and email format	CHECK

2 ER Model

The model has seven strong entity types and one weak entity type, Listening_History. Its relationships are summarised below.

Relationship	Entities	Cardinality	Participation and attributes
Releases	Artist, Album	1:N	Album total, Artist partial
HasTrack	Album, Song	1:N	Song total, Album partial
Performs	Artist, Song	M:N	Song total (BR3), Artist partial; attribute `role`
Manages	Users, Artist	1:1	Both partial
Owns	Users, Playlist	1:N	Playlist total, Users partial
InPlaylist	Playlist, Song	M:N	Both partial; attribute `added_date`
Likes	Users, Song	M:N	Both partial; attribute `liked_date`
Subscribes	Users, Subscription	1:1	Subscription total, Users partial
OfPlan	Subscription, Plan	N:1	Subscription total, Plan partial
Listens (identifying)	Users, Song, Listening_History	1:1:N	Listening_History total, owners partial

A single play has nothing unique about it on its own; it only makes sense as "this user played this song at this moment". We therefore model Listening_History as a weak entity owned by both Users and Song through the ternary identifying relationship Listens, with `listened_at` as its partial key. The same user can play the same song many times, which is why the weak entity side has cardinality N. Its full key is (`user_id`, `song_id`, `listened_at`), and an event disappears if either of its owners is deleted.

`play_count` is shown as a derived attribute because it can always be computed from Listening_History. We still store it, because charts sort by it constantly, and trigger T3 keeps it correct.

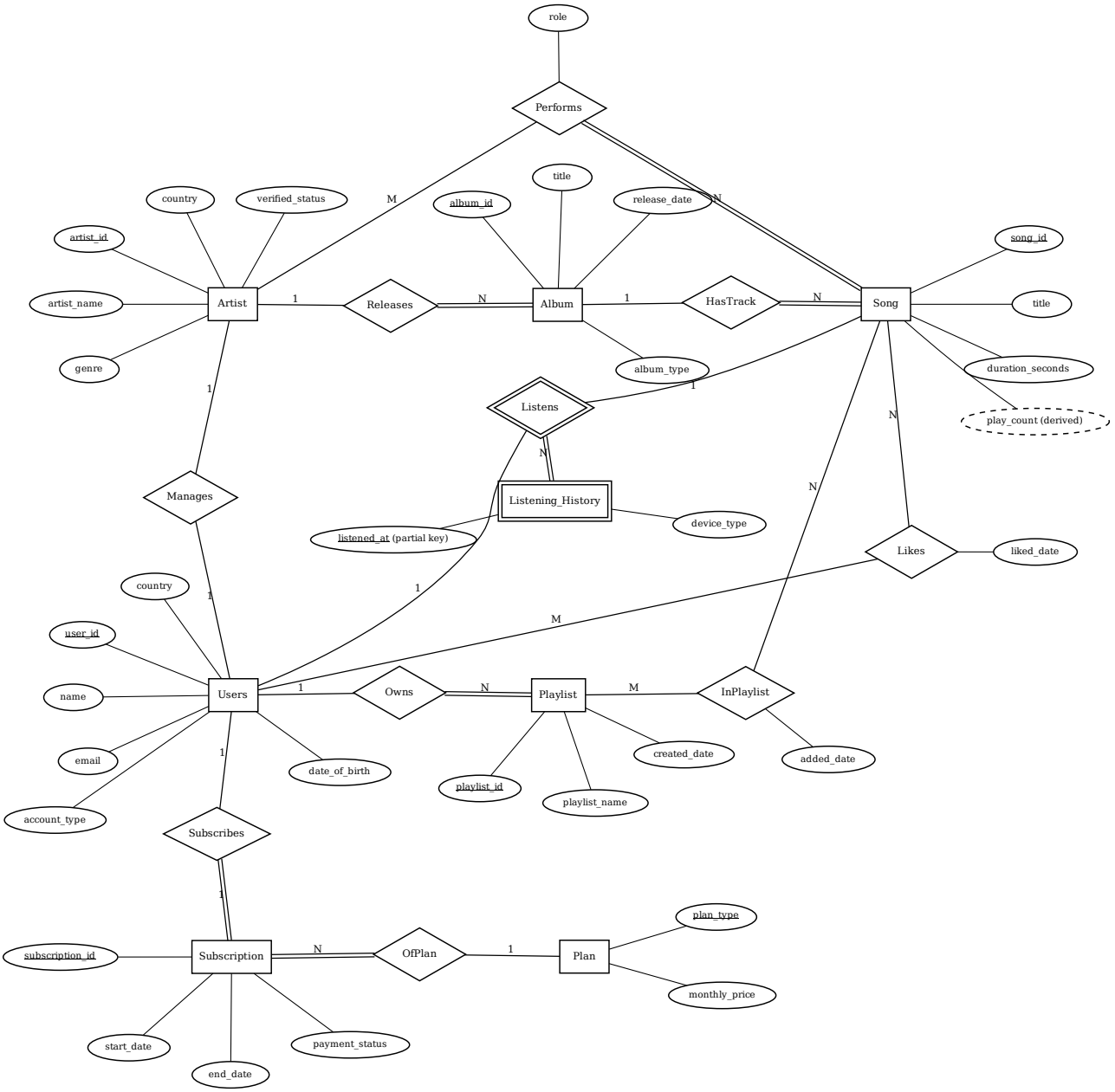


Figure 1: ER diagram in Chen notation. Double rectangles and double diamonds mark the weak entity and its identifying relationship, double lines mark total participation, underlined attributes are keys and the dashed ellipse is a derived attribute.

3 Relational Schema

The ER model was mapped to relations using the standard ER-to-relational mapping steps. Each strong entity became a relation with its own primary key. Listening_History, being weak, takes the keys of its two owners and adds its partial key. For every 1:N relationship the foreign key goes on the N side. The three M:N relationships (Performs, InPlaylist and Likes) each became a relation of their own, keyed by the two participating keys and holding the relationship attribute. For the two 1:1 relationships we used the foreign key approach: Subscribes puts `user_id` in SUBSCRIPTION, the side with total participation, and Manages puts `artist_id` in USERS, with a UNIQUE constraint in both cases. The result is eleven relations, shown in Figure 2 with primary keys underlined and an arrow from each foreign key to the primary key it references. `USERS.auth_id` also references the `auth.users` table that Supabase manages, which is not part of our schema and is not drawn.

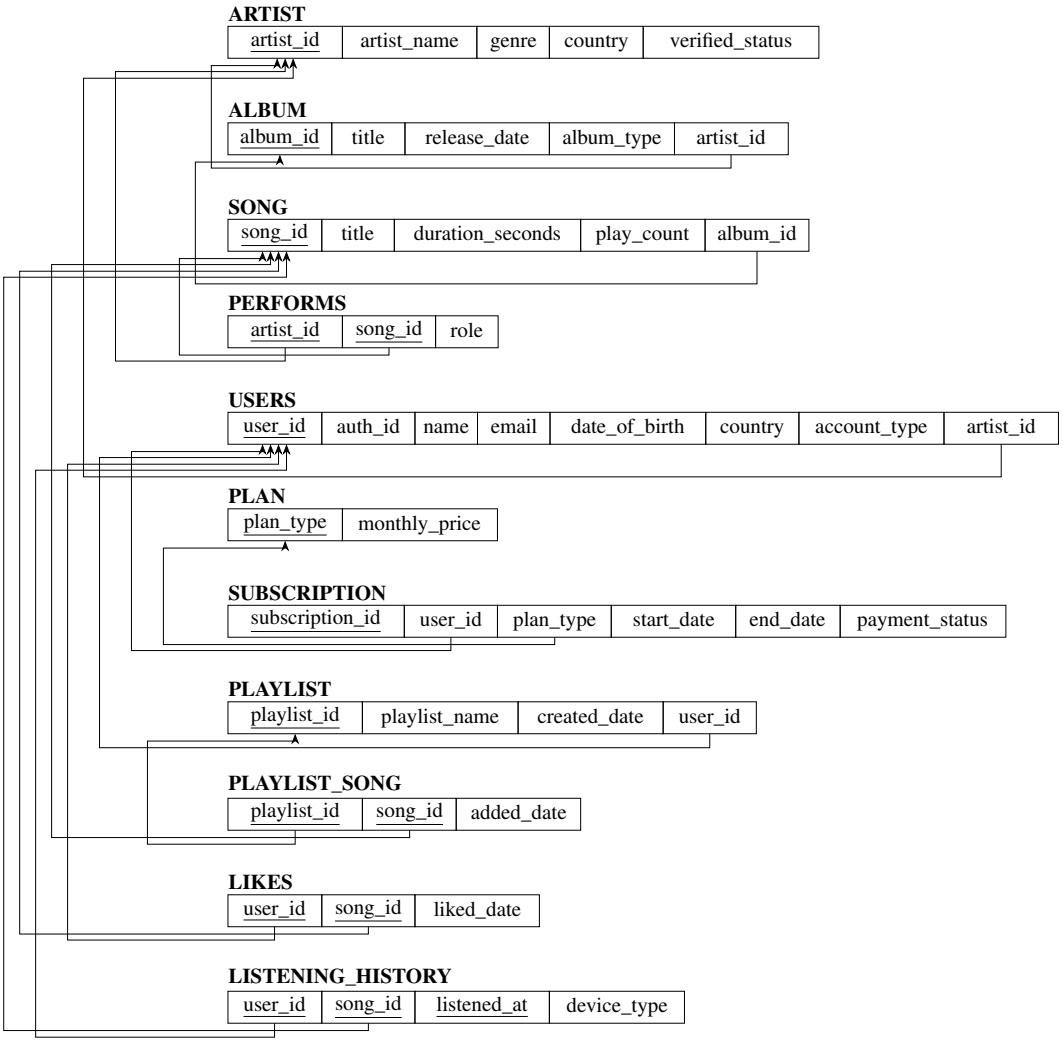


Figure 2: Relational database schema with referential integrity constraints.

Attribute domains and constraints are summarised below (PostgreSQL on Supabase; `SERIAL` surrogate keys; PK primary key, FK foreign key, NN NOT NULL).

Relation	Attributes, domains and constraints
ARTIST	artist_id SERIAL PK; artist_name VARCHAR(150) NN UNIQUE; genre, country VARCHAR(100) NN; verified_status BOOLEAN NN default FALSE
ALBUM	album_id SERIAL PK; title VARCHAR(200) NN; release_date DATE NN; album_type VARCHAR(10) NN in ('album','single','ep'); artist_id INT NN FK → ARTIST ON DELETE RESTRICT
SONG	song_id SERIAL PK; title VARCHAR(200) NN; duration_seconds INT NN between 10 and 3600; play_count INT NN default 0, ≥ 0; album_id INT NN FK → ALBUM ON DELETE CASCADE
USERS	user_id SERIAL PK; auth_id UUID NN UNIQUE FK → auth.users ON DELETE CASCADE; name VARCHAR(150) NN; email VARCHAR(200) NN UNIQUE, email pattern; date_of_birth DATE NN; country VARCHAR(100) NN; account_type VARCHAR(10) NN in ('listener','artist','admin'); artist_id INT UNIQUE FK → ARTIST ON DELETE SET NULL, with CHECK ((account_type = 'artist') = (artist_id IS NOT NULL))
PLAN	plan_type VARCHAR(20) PK ('Premium', 'Family'); monthly_price NUMERIC(6,2) NN > 0
SUBSCRIPTION	subscription_id SERIAL PK; user_id INT NN UNIQUE FK → USERS ON DELETE CASCADE; plan_type VARCHAR(20) NN FK → PLAN ON DELETE RESTRICT; start_date, end_date DATE NN with end_date > start_date; payment_status VARCHAR(10) NN default 'Pending' in ('Paid','Pending','Failed')
PLAYLIST	playlist_id SERIAL PK; playlist_name VARCHAR(200) NN, UNIQUE with user_id; created_date TIMESTAMPTZ NN default now(); user_id INT NN FK → USERS ON DELETE CASCADE
PERFORMS	PK (artist_id, song_id), both INT FK ON DELETE CASCADE; role VARCHAR(20) NN in ('Lead','Featured')
PLAYLIST_SONG	PK (playlist_id, song_id), both INT FK ON DELETE CASCADE; added_date TIMESTAMPTZ NN default now()
LIKES	PK (user_id, song_id), both INT FK ON DELETE CASCADE; liked_date TIMESTAMPTZ NN default now()
LISTENING_ HISTORY	PK (user_id, song_id, listened_at); user_id, song_id INT FK ON DELETE CASCADE; listened_at TIMESTAMPTZ NN default now(); device_type VARCHAR(20) NN in ('Mobile','Desktop','Web','Tablet','Smart Speaker')

4 Integrity Constraints

Every relation has a primary key, and UNIQUE constraints enforce the other candidate keys. Each foreign key declares its delete action: CASCADE where a child means nothing without its parent (a song without its album, a playlist without its owner),

RESTRICT for an artist who still has albums or a plan still in use, and SET NULL for `USERS.artist_id` so a person's account survives the removal of an artist profile. Total participation appears as NOT NULL foreign keys, and every enumerated or bounded attribute has a CHECK. Row Level Security limits each listener to their own data and each artist to their own catalogue.

Rules that span several rows or relations are enforced by triggers.

Trigger	Fires on	What it does
T1 `trg_validate_listen`	LISTENING_HISTORY, before insert	Rejects a listen dated before the album's release or after the current time (BR4)
T2 `trg_validate_playlist_add`	PLAYLIST_SONG, before insert	Rejects an entry dated before the playlist's creation (BR5)
T3 `trg_play_count`	LISTENING_HISTORY, after insert or delete	Adds or subtracts one from the song's play count (BR6)
T4 `trg_validate_dates`	USERS and SUBSCRIPTION, before insert or update	Rejects users younger than 13 and subscriptions starting in the future (BR7)
T5 `trg_playlist_limit`	PLAYLIST, before insert	Rejects a sixth playlist unless the owner has a paid subscription that has not expired (BR8)

T4 is a trigger rather than a CHECK because the rule depends on today's date, and a CHECK constraint should not. BR3 is harder: it asks for total participation of Song in an M:N relationship, which a foreign key cannot express. We handle it by inserting songs only through the stored function `release_album()`, which adds the album, its songs and their performer credits in a single transaction and aborts the whole release if any song lacks a lead performer.

5 Normalization

All attributes are atomic, so every relation is in 1NF. The functional dependencies that hold are listed below. In each relation, every non-trivial dependency has a superkey on its left-hand side, which means every relation is in BCNF and therefore also in 3NF.

Relation	Functional dependencies
ARTIST	artist_id → artist_name, genre, country, verified_status artist_name → artist_id
ALBUM	album_id → title, release_date, album_type, artist_id
SONG	song_id → title, duration_seconds, play_count, album_id
USERS	user_id → every attribute email → user_id auth_id → user_id artist_id → user_id
PLAN	plan_type → monthly_price
SUBSCRIPTION	subscription_id → every attribute user_id → subscription_id
PLAYLIST	playlist_id → playlist_name, created_date, user_id {user_id, playlist_name} → playlist_id
PERFORMS	{artist_id, song_id} → role
PLAYLIST_SONG	{playlist_id, song_id} → added_date
LIKES	{user_id, song_id} → liked_date
LISTENING_HISTORY	{user_id, song_id, listened_at} → device_type

Songs do not carry their own release date, because it is already determined by the album. Plan prices live in a separate PLAN relation; keeping a price in SUBSCRIPTION would introduce the transitive dependency plan_type → monthly_price. An artist's monthly listener count changes every day, so it is computed by a view instead of being stored. The one deliberate exception is `play_count`. It does not break BCNF inside SONG, but it repeats information that Listening_History already holds. Chart queries read it far more often than it changes, and T3 keeps the two consistent.

6 Design Decisions

Performs links artists to songs rather than albums, because collaborations happen on individual tracks. Subscription stays a separate 1:1 relation because most users are on the free tier and would otherwise carry NULL columns in USERS; the UNIQUE `user_id` keeps it 1:1. Passwords are never stored by us: Supabase Auth manages credentials in `auth.users`, which `USERS.auth_id` references. Strong entities use surrogate keys since names repeat or change, while PLAN keeps its small, stable natural key. Since PostgreSQL does not index foreign keys automatically, we index ALBUM(artist_id), SONG(album_id), PLAYLIST(user_id), PERFORMS(song_id), LISTENING_HISTORY(song_id, listened_at) for charts, and lower(SONG.title) for search.

PART B: APPLICATION DESIGN

7 Technology Stack and Architecture

The front end is built with React 19 and Vite 7 in JavaScript, using React Router, Tailwind CSS, Recharts for charts and the supabase-js client for data access. Supabase acts as the backend, providing authentication, an auto-generated REST API, Row Level Security, PL/pgSQL functions and SQL views over a hosted PostgreSQL database. All database objects are kept as ordered SQL scripts in the GitHub repository.

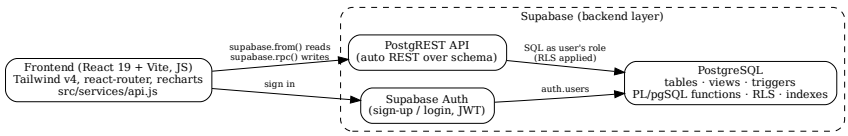


Figure 3: High-level architecture: front end, Supabase backend and PostgreSQL database.

Simple reads go through `supabase.from(...).select()` on tables and views, filtered by Row Level Security. Any write that touches several relations or enforces a business rule is a PL/pgSQL function called with `supabase.rpc()`, so it runs as one transaction: `release_album`, `record_listen`, `add_to_playlist` and `subscribe`. All reports are computed in SQL.

8 Application Workflows

Workflow	What happens	DBMS concepts
W1. Artist releases an album	`release_album()` inserts the album, its songs and their credits; an invalid track or a missing lead performer rolls back the whole release.	Transaction, stored function, CHECK
W2. Listener plays songs and views charts	`record_listen()` inserts the event, T1 validates it and T3 updates the play count; the Charts and Wrapped pages read SQL views.	Triggers, views, CTEs, window functions
W3. Listener manages playlists and subscription	A duplicate song fails on the primary key and a free user's sixth playlist is rejected by T5 until they subscribe; deleting a playlist cascades to its entries.	Keys, trigger, cascading delete, RLS

9 Planned Queries and DBMS Features

#	Query	Techniques
Q1	Top 10 songs in each country over the last 30 days	CTE, joins, GROUP BY, RANK() OVER (PARTITION BY)
Q2	Monthly listeners of each artist	Joins, COUNT(DISTINCT), view
Q3	Share of listening by genre over the last 30 days	Joins, GROUP BY, SUM() OVER ()
Q4	A listener's Wrapped: top artist, top song, minutes listened, top genre	Several CTEs, aggregation
Q5	Monthly revenue by plan	Join with PLAN, aggregation, date filter
Q6	Artists with the most distinct collaborators	Self-join on PERFORMS
Q7	Songs a listener has played at least five times but not liked	GROUP BY, HAVING, NOT EXISTS
Q8	Songs played more often than the average song on their album	Correlated subquery

DBMS features used: transactions in the four stored functions, triggers T1 to T5, views for the reports, the indexes in Section 6, and Row Level Security. Seed scripts will load well over 500 records, mostly listening history.

10 Progress and Team Contributions

So far we have completed the ER model, the mapping to the relational schema described in Part A, and the scoping of the technology stack and application architecture described in Part B.

Member	Contribution
Ritwik Sethi	Requirements analysis; ER model (entities, relationships, ER diagram); relational schema (mapping, schema diagram)
Arihant Rakhecha	ER model (weak entity, participation constraints); relational schema (domains, keys); integrity constraints and trigger design
Shreyas Ghosh	Normalization (functional dependencies, BCNF); application design (architecture, workflows)
Shreya Agrawal	Application design (technology stack, planned queries); documentation (design report)
Piyush Gaikwad	Normalization (redundancy, derived attributes); documentation (README, GitHub repository)

GitHub repository: `https://github.com/omnomnomm/music-streaming-platform`